POSTGRES: ІНДЕКСИ І MVCC

EXPLAIN (ANALYZE, BUFFERS): планер, ціна запиту у 8КБ-сторінках і VACUUM

Заняття №12

Node.js Pro

Лектор
Кирило Орлов

r_d

ДРАБИНКА ДНЯ: SEQ SCAN → B-TREE → ПЛАНЕР → MVCC → VACUUM

Половина I — біль і ліки	Половина II — відмови планера і MVCC
orders: мільйон рядків — і лише PK	селективність 93% · lower() · LIKE '%...'
EXPLAIN (ANALYZE, BUFFERS) — окуляри	MVCC: UPDATE = нова версія рядка
B-tree: 11 346 сторінок → 24	bloat 18→71 МБ · VACUUM · VACUUM FULL
складений · частковий · expression	аудит: pg_stat_user_indexes → DROP

НАСКРІЗНА НИТКА

Одна таблиця orders на 1 000 000 рядків і одне питання: скільки 8КБ-сторінок платить запит. Мілісекунди залежать від машини, buffers — від структури даних. Тому весь вечір дивимось на buffers.

ПЛАН ЗАНЯТТЯ



- Словник дня і мільйон рядків болю
- EXPLAIN (ANALYZE, BUFFERS): читаємо план
- B-tree: простий, складений, частковий
- Коли планер ігнорує індекс — і чому
- MVCC: два термінали, один рядок
- VACUUM, bloat, аудит індексів + ДЗ

СЛОВНИК ДНЯ: ШІСТЬ СЛІВ ЦІЄЇ ЛЕКЦІЇ

01. сторінка

8 КБ — одиниця обміну з диском; buffers у плані рахують саме їх

02. планер

мозок: обирає план за вартістю, дивлячись у статистику ANALYZE

03. селективність

частка таблиці, що пройде фільтр: 0,002% — індекс, 93% — Seq Scan

04. buffers

скільки сторінок торкнувся запит — головна цифра сьогодні

05. xmin / xmax

невидимі колонки кожного рядка: хто версію створив і хто замінив

06. bloat

мертві версії рядків у файлі; прибирає VACUUM, стискає FULL

БІЛЬ: ОДИН WHERE ЧИТАЄ ВСЮ ТАБЛИЦЮ

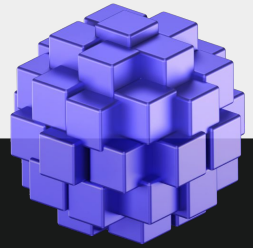
orders: 1 000 000 рядків, 89 МБ = 11 346 сторінок

запит: «замовлення користувача 4242» — це 21 рядок

Seq Scan читає всі сторінки: Rows Removed ≈ мільйон

індексу немає — Postgres нема за що вхопитись

11 346 сторінок заради 21 рядка — ось що лікуємо. Але спершу навчимось це БАЧИТИ: EXPLAIN на наступному слайді.



```
ТАБЛИЦЯ = КУПА СТОРІНОК

orders · heap · 89 МБ

| p0 | p1 | p2 | … 11346 |

кожна сторінка = 8 КБ

WHERE user_id = 4242

Seq Scan: p0 → p11345
  прочитано: 11 346 стор.
  знайдено:  21 рядок

селективність 0,002% —
а прочитано 100% таблиці
```

EXPLAIN (ANALYZE, BUFFERS): ОКУЛЯРИ ЛЕКЦІЇ

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders
WHERE user_id = 4242;

Gather (rows=21)
  Workers Planned: 2
  Workers Launched: 2
  Buffers: shared hit=11346
  -> Parallel Seq Scan on orders
       Filter: (user_id = 4242)
       Rows Removed by Filter: 333326
         (loops=3 → ≈ мільйон)
       Buffers: shared hit=11346

Execution Time: 17.805 ms
```

шлях запиту: parser → planner → executor

1 · план — дерево: читаємо з найглибшого вузла

2 · cost — прогноз планера, actual — факт

3 · Rows Removed — прочитав і викинув

4 · Buffers — реально торкнуті сторінки

урок: EXPLAIN без ANALYZE — лише прогноз

«ПОКАЖЧИК У КНИЗІ»: ІНДЕКС ЯК АНАЛОГІЯ

ЧАСТИНА POSTGRES	АНАЛОГІЯ (КНИГА)
• таблиця (heap) — сторінки підряд • B-tree — окремий покажчик • лист індексу → ctid рядка • Seq Scan — гортати все підряд • CREATE INDEX — скласти покажчик	• книга на 11 346 сторінок • алфавітний покажчик наприкінці • «acid — див. с. 314» • читати від обкладинки до обкладинки • один раз сісти й виписати терміни

Де аналогія ламається: покажчик друкують раз і назавжди, а B-tree оновлюється при КОЖНОМУ INSERT і UPDATE. Індекс не безкоштовний: за нього платить кожен запис.



B-TREE: ВІДСОРТОВАНЕ ДЕРЕВО, НЕ МАГІЯ

B-tree — відсортоване дерево «ключ → ctid»

мільйон ключів — глибина ≈ 3 рівні

уміє: «=», діапазони, ORDER BY без Sort

по ctid далі йдемо в heap по сам рядок

не вміє: LIKE '%...' — префікс невідомий

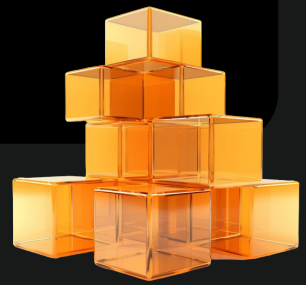
GIN/GiST/BRIN — інші структури й задачі

```
    ПОШУК user_id = 4242

       [ корінь ]
      /     |     \
 [гілка] [гілка] [гілка]
            ↓
 [ лист: 4240 4241 4242* ]
       * → ctid (стор.17, №3)
            ↓
 heap: сторінка 17, рядок 3

3 стрибки замість 11 346 сторінок

ключі відсортовані, як словник:
 'User4242%…' → старт відомий
 '%4242@…'    → старту немає
```





```
-- Крок 3: лікуємо той самий запит (03-btree.sql)
CREATE INDEX idx_orders_user_id ON orders (user_id);
ANALYZE orders;                        -- свіжа статистика для планера

EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM orders WHERE user_id = 4242;

 Bitmap Heap Scan on orders (actual rows=21)
   Recheck Cond: (user_id = 4242)
   Buffers: shared hit=21 read=3               ← було 11 346
   ->  Bitmap Index Scan on idx_orders_user_id
         Index Cond: (user_id = 4242)          ← умова В ІНДЕКСІ, не у Filter
 Execution Time: 0.100 ms                      ← було 17.805 ms

-- ціна: окрема структура на диску (сама таблиця — 89 MB)
SELECT pg_size_pretty(pg_relation_size('idx_orders_user_id'));   -- 7800 kB
```

```
-- Крок 4: адмінка — «pending за останні 30 днів» (04-composite.sql)
-- правило: рівність — ліворуч, діапазон — праворуч
CREATE INDEX idx_orders_status_created ON orders (status, created_at);

EXPLAIN (ANALYZE, BUFFERS) SELECT count(*) FROM orders
WHERE status = 'pending' AND created_at >= now() - interval '30 days';

 Index Only Scan using idx_orders_status_created
   Index Cond: ((status = 'pending') AND (created_at >= …))
   Heap Fetches: 0                              ← у таблицю не лізли взагалі
   Buffers: shared hit=4 read=8                 ← 12 сторінок
 Execution Time: 0.277 ms

-- антиприклад: (created_at, status) — діапазон першою колонкою. Щоб планер узяв
-- саме його, «правильний» ховаємо: BEGIN; DROP INDEX …; ROLLBACK
 Index Only Scan using idx_orders_created_status
   Buffers: shared hit=1 read=203               ← 204 сторінки, той САМИЙ запит
 Execution Time: 1.591 ms
```



СКЛАДЕНИЙ ІНДЕКС: ПОРЯДОК КОЛОНОК — КОНТРАКТ

правило: рівність — ліворуч, діапазон — праворуч: (status, created_at)

той самий запит: правильний порядок — 12 buffers, перевернутий — 204

ORDER BY created_at DESC зник як вузол Sort — індекс уже віддає впорядковано

бонус: лівий префікс працює сам — запит по одному status теж накритий. Тримати поруч (status) і (status, created_at) — дубль





```
-- Крок 5 (05-partial.sql): 93% рядків — completed,
-- а вся адмінка живе у pending. Нащо індексувати 93%?
CREATE INDEX idx_orders_pending ON orders (created_at)
WHERE status = 'pending';                    -- індексуємо лише 5% таблиці

SELECT pg_size_pretty(pg_relation_size('idx_orders_status_created')); -- 38 MB
SELECT pg_size_pretty(pg_relation_size('idx_orders_pending'));      -- 1112 kB

-- складений на час цього EXPLAIN схований (BEGIN; DROP INDEX …; ROLLBACK) —
-- інакше планер бере ЙОГО. Той самий запит, та сама швидкість, індекс у ~35× менший:
 Limit
   ->  Index Scan Backward using idx_orders_pending on orders
         Index Cond: (created_at >= (now() - '30 days'::interval))
 Execution Time: 0.055 ms

-- зворотний бік: для status = 'cancelled' цього індексу НЕ ІСНУЄ —
-- планер бере частковий, лише коли WHERE запиту дослівно покриває WHERE індексу
```

ПЛАНЕР КАЖЕ «НІ»: ТРИ ВИПАДКИ (КРОК 6)

93% таблиці

completed — це 93%: стрибати індексом по 930 тис. рядків дорожче, ніж Seq Scan підряд (≈40 мс)

функція над колонкою

у B-tree лежить 'User42@...', а не lower(). ≈73 мс Seq Scan → expression-індекс: 0.07 мс

LIKE '%...'

'%42@...' — % на початку: префікса немає, дереву нема звідки почати. LIKE живе у Filter, не в Index Cond

Сигнал у EXPLAIN

Індекс — пропозиція, а не наказ: планер порівнює вартість планів у сторінках і бере найдешевший. Умова в рядку Filter замість Index Cond = «індекс є, але не працює». Це перше, що шукаємо в повільному запиті.

SEQ SCAN ПРОТИ INDEX SCAN: ЧЕСНА ТАБЛИЦЯ

Аспект	Seq Scan	Index Scan
Як читає	усі сторінки таблиці підряд	стрибками лише потрібні сторінки
Вигідно, коли	фільтр пропускає десятки відсотків рядків	одиниці відсотків і менше
Ціна запису	нульова — таблиця і так пишеться	оновлення на кожен INSERT/UPDATE
ORDER BY	потрібен окремий вузол Sort	віддає вже впорядковано
Сигнал у плані	Filter + Rows Removed by Filter	Index Cond / Index Only Scan

MVCC: UPDATE НЕ ПЕРЕПИСУЄ РЯДОК

01. кожен рядок має службові колонки: ctid, xmin, xmax

02. UPDATE = INSERT нової версії + xmax на старій

03. читачі не блокують писачів — у кожного свій знімок

04. READ COMMITTED: знімок береться на кожен запит

05. стара версія лежить у файлі мертвою — до VACUUM

ДВА ТЕРМІНАЛИ, ОДИН РЯДОК: MVCC НАЖИВО

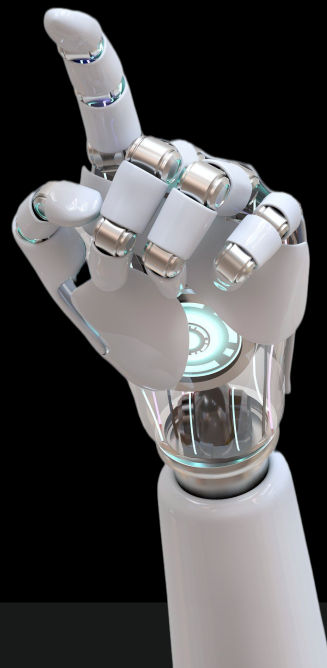
```
СЕАНС A (пише)                                  СЕАНС B (читає)
──────────────────────────────────────────────────────────────────

SELECT ctid, xmin, xmax, status …
→ (0,1)  xmin=733  xmax=0  completed

BEGIN;
UPDATE orders SET status='cancelled'
       WHERE id = 1;                            SELECT … WHERE id = 1;
→ (11345,73)  xmin=758  cancelled               → (0,1)  xmax=758  completed
  нова версія на новій сторінці                   СТАРА версія, БЕЗ очікування

COMMIT;                                         SELECT … WHERE id = 1;
                                                → (11345,73)  xmin=758  xmax=0
                                                  cancelled — нова версія

ctid змінився — рядок переїхав; стара (0,1) лежить мертвою до VACUUM (крок 8)
```



СЛУЖБОВІ КОЛОНКИ: ПАСПОРТ ВЕРСІЇ РЯДКА

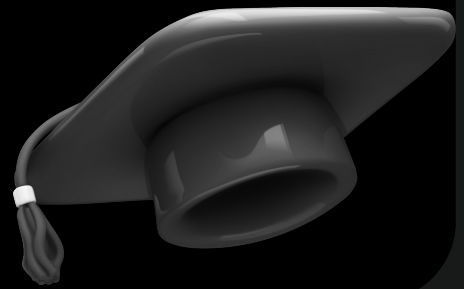
Колонка	**Що означає**	**У демо (крок 7)**
ctid	фізична адреса версії: (сторінка, слот)	(0,1) → (11345,73): рядок фізично переїхав
xmin	id транзакції, що створила версію	нова версія: xmin = 758 (транзакція A)
xmax	хто позначив на заміну; 0 — версія жива	стара: xmax = 758, нова: xmax = 0
снепшот	набір версій, який бачить запит	B читає стару версію — без очікування
блокування	писач блокує лише писача того ж рядка	SELECT у B не чекав UPDATE в A ані мс

```
-- Крок 8: три UPDATE по 200 000 рядків — файл виріс учетверо (08-vacuum.sql)
CREATE TABLE bloaty (LIKE orders) WITH (autovacuum_enabled = off);
INSERT INTO bloaty SELECT * FROM orders WHERE id <= 200_000;    -- 18 MB

UPDATE bloaty SET amount = amount + 1;    -- ×3: кожен прохід = нова версія
                                          --     КОЖНОГО рядка
SELECT pg_size_pretty(pg_relation_size('bloaty'));        -- 71 MB
SELECT n_dead_tup FROM pg_stat_user_tables
 WHERE relname = 'bloaty';                                -- ≈599 200 мертвих

VACUUM bloaty;     -- мертві прибрано, місце — під повторне використання
                   -- файл ЛИШИВСЯ 71 MB, n_dead_tup = 0

VACUUM FULL bloaty;    -- переписує таблицю: знову 18 MB, але тримає
                       -- ACCESS EXCLUSIVE lock — не читає ніхто
```



VACUUM ПРИБИРАЄ, АЛЕ НЕ СТИСКАЄ

VACUUM прибирає мертві версії — місце для нових рядків

але файл НЕ зменшується: 71 МБ лишилися 71 МБ

VACUUM FULL: знову 18 МБ, але ACCESS EXCLUSIVE lock

autovacuum будиться сам: поріг = 50 + 20% живих рядків

у compose лекції naptime = 10 с — лише задля демо

росте n_dead_tup — сигнал: autovacuum не встигає

«ГОТЕЛЬ І ПОКОЇВКА»: MVCC ЯК АНАЛОГІЯ

ЧАСТИНА POSTGRES	АНАЛОГІЯ (ГОТЕЛЬ)
• версія рядка • UPDATE — нова версія • мертві версії (bloat) • VACUUM • VACUUM FULL	• номер у готелі • гостя переселяють у новий номер • старі номери стоять «зайняті» привидами • покоївка готує номери новим гостям • закрити готель і перебудувати менший

ЩО СВІДОМО ЛИШАЄМО ЗА ДУЖКАМИ

GIN · GiST · BRIN · hash — сьогодні лише «коли глянути»: jsonb і повнотекст — GIN, геометрія й діапазони — GiST, гігантські append-only — BRIN

рівні ізоляції, аномалії, FOR UPDATE, SKIP LOCKED, deadlock'и — уся L#14. Тут лише фундамент: MVCC і дефолтний READ COMMITTED

ORM — наступна L#13: TypeORM піднімає СВІЙ Postgres (lecture13, порт 21300), з нашим 21200 не конфліктує. Пул і репліки — L#15

назад, до L#11: пароль до бази лежить у compose відкрито НАВМИСНО — сьогодні між тобою і планером не стоїть ані ORM, ані конфіг-модуль



```
-- Фінал кроку 8 (демо C): аудит — хто з індексів не окупається?
SELECT indexrelname AS index, idx_scan AS scans,
       pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes WHERE relname = 'orders' ORDER BY idx_scan;

            index           | scans |  size
 ---------------------------+-------+---------
  idx_orders_amount         |     0 | 21 MB      ← «про запас» із кроку 3
  idx_orders_email          |     0 | 8704 kB    ← замінив expression-індекс
  idx_orders_email_lower    |     1 | 8704 kB
  idx_orders_status_created |     1 | 38 MB
  idx_orders_pending        |     1 | 1120 kB
  idx_orders_user_id        |     7 | 7800 kB
  orders_pkey               |     8 | 21 MB      ← тримає PK, не чіпати

DROP INDEX IF EXISTS idx_orders_amount;  -- 21 МБ диска й гальмо кожного
                                         -- INSERT за нуль прочитаних рядків
```

ПІДСУМКИ + МІСТ ДО L#13 І L#14

ІНДЕКС

окрема структура зі своєю ціною; планер може її й не взяти

EXPLAIN

buffers важливіші за мілісекунди; Filter замість Index Cond — сигнал тривоги

ДАЛІ

L#13 TypeORM — ті самі запити, свій Postgres на 21300 · L#14 — ізоляція й локи поверх MVCC

ДЗ #12

10 балів (Heavy) · дедлайн 08.09 · схема БД курсового + seed ≥100 тис. · 3 запити з EXPLAIN до/після · partial або expression · свіжий клон піднімається

MVCC

UPDATE = нова версія; мертві прибирає VACUUM, стискає лише VACUUM FULL

Q&A

???

r_d

ЗАВЖДИ Є КУДИ ЗРОСТАТИ